

Where Does an AI Learn a New Rule?

ADAPT — a compact, evidence-backed walkthrough of inference-time adaptation. All numbers below are our own measured results (provenance in ``research/experiments/*/results.json``), unless labelled PUBLISHED.

Problem

Machine-learning systems are usually frozen at deployment: they predict well on what they were trained on and fail on everything else. Humans do not work that way. Given a handful of examples of an **unseen** rule — a spelling convention, a fresh procedure — we adjust on the spot, without re-training a brain. Why can't a model do the same?

The conventional reflex is "test-time training": run gradient updates on the parameters at inference time. But gradients are expensive, fragile, and they overwrite what the model already knows. The deeper question is not "how do we update it?" but:

Where is the new task information stored, how is it updated, and what does that choice cost?

Insight

Three places can carry a new rule:

- **Context** — the demonstration examples are handed to the model alongside the query. No update at all: the readout pattern-matches against the examples.
- **Parameters** (test-time adaptation, TTA) — the demonstration set drives gradient updates to the weights θ before readout.
- **State** — the recurrent-network middle path: persistent weights θ stay fixed ($\Delta\theta = 0$) while an internal adaptive state is updated per example (e.g. $s \leftarrow s + \phi(x, y)$), and the readout of **that** state solves the query.

The three substrates trade off ***capacity, cost, retention, and interference***. State adaptation is the interesting middle: the new rule lives in a finite register the network rewrites at inference time.

Experiments

We built controlled toy implementations over synthetic task families —

linear, symbolic, compositional, and an ARC-like grid proxy — with exact hidden ground truth, 200 paired runs per cell, and Wilson 95% CIs. The full sweep matrix is frozen (D5.1) and every payload carries a provenance envelope: seed, split, generator version, model version, and config hash. Nothing is simulated to look good: what you see is the actual engine (``src/adapt/``) running over HTTP.

Results

Acquisition. On unseen continuous linear rules, frozen accuracy is 0.11; state adaptation reaches 1.00 with $\Delta\theta = 0$ and $\Delta s > 0$ (per-demonstration state update). Context reaches 1.00 too — pattern matching is enough here. Gradient-based TTA reaches only 0.255 on the same sweep. On symbolic and ARC-like proxy families, frozen = 0.00 and state = 1.00.

Causality, not decoration. The state is *necessary*: zeroing or swapping the adapted state collapses accuracy from 1.00 to 0.07 (zero / 0.05–0.07 across all perturbation modes), restoring it returns accuracy to 1.00. McNemar paired $p < 10^{-2}$; Holm-corrected rejection in every mode. This is direct evidence the behavior lives in the state.

Interference. $A \rightarrow B \rightarrow A$: state's retention on task A drops to 0.10 after B overwrites the shared register; context keeps A fully (delivers the examples each query) and is literally stateless. This is "catastrophic interference" reproduced in exact arithmetic.

Capacity. The register is a bottleneck: compressing s past the sufficient statistics destroys accuracy as a **cliff**, not a graceful slope. More test-time compute helps gradient readers only with diminishing returns.

Learned variants — honest negatives. Under our meta-training budget, the learned-recurrent checkpoint did *not* acquire the linear rule ($0.09 \approx 0.11$ frozen; learned TTA 0.26, learned TTT 0.445). We publish these negatives; the demonstrated 1.00 results come from the analytical state substrate, not from overfitting a checkpoint.

BDH / BDH-CQ frontier

BDH (Kosowski, Uznański, Chorowski & Stamirowska, 2025, arXiv:2509.26507)

proposes a post-Transformer architecture whose *working memory lives in a synaptic state* σ , updated Hebbian-style ($\sigma \leftarrow \sigma + \eta XY$) and separate from persistent parameters. BDH-CQ (Engdahl, Kosowski, Chorowski & Stamirowska, 2026, arXiv:2608.09888) extends the envelope: demonstrations write to recurrent memory, and a query is solved by latent iterative computation with fixed parameters.

The abstract envelope — *demonstrations write to state, readout of state solves the query, parameters fixed* — is exactly what our transparent toy substrate isolates and intervenes on. We probed the official implementations ourselves (R1/R2, `research/bdh/reproductions/`): the path exists and the memory-write switch threads through without error. Their headline numbers — BDH-CQ's 150M-parameter / 29.5% pass@2 — were **not reproduced by us** and stay cited as PUBLISHED with a scope guard; we never present them as our own result, and the UI's "Grid Transformation Lab" label never claims ARC Reasoning.

Limitations

- Toy scale, exact ground truth, synthetic families. No claim is made past these systems.
- The learned-recurrent negatives show where our budget ran out — a real limit of the study, reported rather than hidden.
- The ARC-like proxy pool is small and is a synthetic grid proxy, not ARC-AGI.
- Learner-study learning-gain: n=10 distinct participants to date (target 8–15); results are preliminary (mean gain +1.2/9, transfer to the unseen challenge 2/8) — the honest, still-open question.

Interactive artifact

The laboratory lets you watch the state change per demonstration, break it with interventions, induce interference, and compare context vs parameters vs state on identical episodes — with every number live or explicitly labelled precomputed/published, and exact ground truth beside every estimate.

*Repository: <https://github.com/Adithkp03/ADAPT> · full provenance and methodology in-repo. This post is a narrative snapshot of the measured

record, not marketing.*